



NEXTGEN BUILDER INITIATIVE

presented by Aadhirai Innovations

SESSION 01 · LIVE WEBINAR · 90 MINUTES · NO LAPTOP NEEDED

One Requirement. 180 Days.

The true story of how software actually gets built

We follow one billing system from the sentence a client says on a Monday morning to the day somebody else takes over running it. At every stage: what your options are, how to choose between them, and exactly where AI helps — and where it quietly lies to you.



Monday, 9:40 a.m.

“We have 40 stores. Billing is on an old system that keeps crashing at month end. We need a new one. Make a bill, apply the right price and tax, print it, keep the record, handle returns. Nothing fancy. Can you do it in six months?”

That is the entire requirement. It never changes for the rest of this session. Everything else does.

Sri Krishna Stores

40 outlets across Tamil Nadu. 12,000 bills a day.
Triple that in Deepavali week.

The team

You, a tech lead, three engineers, a QA, and a manager who wants dates by Friday.

The budget

Fixed. The deadline was decided before anyone knew what the system was.

Six months from now this will either be running in 40 stores, or it will be a story people tell about a project that failed. The difference is decisions, not effort.

Eight stages. 180 days. One requirement.



Day 1–12	Gather	Find out what they actually need — not what they said
Day 13–25	Design	Write it down so it can be built and argued with
Day 26–35	Tech	Pick a language and a database, for reasons that will last 8 years
Day 36–50	Architect	Decide the shape. The most misunderstood decision in software
Day 51–130	Build	Where AI has changed the job the most, and not in the way people think
Day 90–150	Test	Which tests, at which stage, and the one that catches disasters
Day 151–165	Deploy	Getting it to 40 stores without breaking any of them
Day 166+	Run	Monitoring, the 9 p.m. phone call, and handing it over



NEXTGEN BUILDER INITIATIVE

presented by Aadhirai Innovations



DAY 1 — 12 · 00:08 — 00:22

Gather

Finding out what they actually need, not what they said

IN THIS STAGE

Five ways to gather

When to use which

What watching found

AI at this stage

1

Gather

2

Design

3

Tech

4

Architect

5

Build

6

Test

7

Deploy

8

Run

Five ways to find out. Each finds something different.

DAY 2 · THE TEAM SITS DOWN

“So... do we just email them a list of questions?” Everyone looks at the tech lead. There are five real techniques and most engineers know only one of them.

Interview

Prepared questions, one person at a time, written down.

CHOOSE WHEN

Always start here. It maps the territory — but people describe the process they think they follow.

Observation

Sit and watch someone bill a customer for half a day. Say nothing.

CHOOSE WHEN

When the work is physical and habitual. Highest value per hour of any technique.

Document study

Collect every bill, register, form and spreadsheet in use today.

CHOOSE WHEN

Always. Old documents contain rules nobody remembers deciding.

Prototype

A clickable mock-up shown before any real code.

CHOOSE WHEN

When the user cannot describe what they want but will recognise it instantly.

Workshop

Get the disagreeing departments in one room, facilitated.

CHOOSE WHEN

When two teams want different things and email has failed for six weeks.

Exception hunting

Ask relentlessly: “when does that NOT happen?”

CHOOSE WHEN

At every stage, free, and it finds more real requirements than anything else.

Half a day at a billing counter

A notebook of special rates for regular customers



Customer-specific pricing exists. It is a whole subsystem. The owner never mentioned it because to him that is just selling.

Roughly 1 bill in 15 is cancelled and reprinted



Cancellation with an audit trail is core, not an edge case. Under GST you cannot reuse a cancelled bill number.

The clerk types a code, not the product name



They have memorised 400 codes. Your beautiful search box will slow them down and they will hate it.

Two staff share one login “because it's faster”



Every per-user audit trail you build is fiction until you fix login friction.

Goods leave before the bill is printed



Delivery and billing are separate events. Modelling them as one breaks the actual business.

None of this was hidden. Nobody asked.

What changes when AI enters requirement gathering

AI IS GENUINELY GOOD AT THIS

- Turning messy meeting notes into a structured document in minutes
- Suggesting edge cases you would not have listed — “what if the customer returns half a batch?”
- Attacking your own draft: “find the gaps in this requirement”
- Translating between how the business talks and how engineers write

WHERE IT QUIETLY GETS IT WRONG

- It fills silence with plausible invention. Ask for requirements and it will confidently produce a load profile, an uptime target and a compliance stance — all reasonable, none verified with anybody
- It cannot sit at a billing counter for four hours. It will never find the notebook
- A polished document feels finished. Confidence is not the same as correctness

SO YOUR JOB BECOMES

Verifying, not writing. AI gets you a full draft in an hour instead of a week — and every assumption inside it still needs a human to confirm with the person paying. The bottleneck moved from typing to checking.



NEXTGEN BUILDER INITIATIVE

presented by Aadhirai Innovations



DAY 13 — 25 · 00:22 — 00:34

Design

Writing it down so it can be built — and argued with

IN THIS STAGE

Functional vs non-functional

Ways to specify

Making it measurable

AI at this stage

1

Gather

2

Design

3

Tech

4

Architect

5

Build

6

Test

7

Deploy

8

Run

What it does, versus how well it does it

DAY 14 · THE QUESTION THAT STOPS THE ROOM

“How fast should the bill print?” Nobody knows. Somebody says “fast”. That word will cost three weeks in month five.

FUNCTIONAL — WHAT IT DOES

The client describes these fluently. This is the easy half.

- Make a bill
- Apply price and tax
- Print or email it
- Handle a return

NON-FUNCTIONAL — HOW WELL

You have to drag every one of these out of somebody.

- 3 seconds, or 30, at the busiest hour?
- How many counters bill at once?
- Must it work when the internet dies?
- How long must bills be kept — legally?

The left column decides what screens you build. The right column decides the entire architecture. Guess which one usually gets skipped.

This is the single highest-leverage idea in the whole session. If you take one thing, take this.

Four ways to specify. They are not interchangeable.

BRD / FSD

A formal document. Sections, sign-off, version numbers.

CHOOSE WHEN

Fixed-price contracts, regulated industries, or when a client will dispute scope later. Slow but defensible.

User stories

“As a billing clerk I want... so that...” on a board.

CHOOSE WHEN

Agile teams, evolving scope, a client available to answer questions weekly.

Prototype as spec

A clickable mock-up is the specification.

CHOOSE WHEN

When the users cannot read a document but will tell you instantly what is wrong with a screen.

Acceptance criteria

Given / when / then, testable, per requirement.

CHOOSE WHEN

Always, alongside whichever of the above you chose. This is what “done” means.

WHAT WE CHOSE, AND WHY

User stories with acceptance criteria — because the client is available and the scope will move. A 60-page FSD would be out of date by month three and nobody would read it twice.

Non-goals — the things you are explicitly NOT building

No accounting

They have a CA with software. We export to it.

No loyalty points

Wanted, but phase 2. Written down with a date.

No mobile app

Counter staff use a keyboard. A phone is slower.

What changes when AI enters design and specification

AI IS GENUINELY GOOD AT THIS

- Generating user stories and acceptance criteria from a rough description, in bulk
- Producing a first-draft data model that is 80% right and saves a day
- Spotting contradictions between requirements you wrote weeks apart
- Rewriting a technical spec into language the client can actually read

WHERE IT QUIETLY GETS IT WRONG

- It will happily invent non-functional numbers. “The system shall support 10,000 concurrent users” — where did that come from? Nowhere
- Generated acceptance criteria test the happy path. The cancelled bill, the half return, the expired batch — those come from humans who have seen them
- A spec that reads beautifully can still describe the wrong system

SO YOUR JOB BECOMES

Deciding, not drafting. The document takes an afternoon instead of two weeks — and every number in it is still a decision somebody has to own and defend.



NEXTGEN BUILDER INITIATIVE

presented by Aadhirai Innovations



DAY 26 — 35 · 00:34 — 00:44

Technology

Picking a language and a database for reasons that last eight years

IN THIS STAGE

Language options

Database options

How to actually choose

AI at this stage

1

Gather

2

Design

3

Tech

4

Architect

5

Build

6

Test

7

Deploy

8

Run

Six real options. All of them would work.

DAY 27 • THE ARGUMENT

It lasts ninety minutes. Somebody says Go is faster. Somebody says Python is quicker to write. Nobody mentions that the developer who maintains this in 2032 has not been hired yet.

Java • Spring

Verbose, mature, enormous hiring pool in India. Runs for years untouched.

CHOOSE WHEN

Business systems that must live 8+ years and change hands. Transactional work.

C# • .NET

Everything Java offers with nicer tooling. Fully cross-platform now.

CHOOSE WHEN

Same cases as Java, especially if the client is already Microsoft-centric.

Python • Django

Fastest to a working system. Free admin screens. Readable by anyone.

CHOOSE WHEN

Speed of delivery matters most, or the work is data-heavy. Weaker on huge business domains.

TypeScript • Node

One language front to back. Biggest package ecosystem. Fast hiring.

CHOOSE WHEN

Full-stack teams, or one developer owning both ends. Watch the dependency churn.

Go

One binary, no runtime, trivial to deploy and operate.

CHOOSE WHEN

Operational simplicity is king, or you are building infrastructure rather than business logic.

PHP • Laravel

Huge Indian SME footprint, cheapest hosting, genuinely good framework.

CHOOSE WHEN

Cost is the binding constraint and the team already knows it. Do not sneer — it ships.

Four questions that decide it. None is about speed.

01 Who can we hire in 2032?

This system outlives its author. If the client cannot find a replacement locally at a sane rate, the technology choice has failed — however elegant it was.

02 What does this team already know?

A team writing their second system in a familiar language beats a team writing their first in a better one. Every time.

03 Does the ecosystem fit the domain?

Ready-made libraries for GST, PDF bills, barcode scanners, thermal printers. A good ecosystem removes weeks.

04 How hard is it to operate?

Fewer moving parts, fewer things to fix at 9 p.m. Nobody puts this on a slide and everybody feels it in year two.

THE REASON THAT SHOULD NOT DECIDE IT

“Which is fastest?” At 12,000 bills a day — and at a million — every option on the previous slide is fast enough. The time goes to the database and the network, not the language. Language speed matters for a small minority of systems. This is not one of them.

We chose Java. Not because it is best — that question has no answer — but because it wins on questions 1 and 3.

Six families. They are built for different things.

Relational

PostgreSQL, MySQL, SQL Server. Tables, relationships, transactions.

CHOOSE WHEN

You have related data and need it to be correct. The right default for business systems.

Document

MongoDB, DynamoDB. Flexible self-contained records.

CHOOSE WHEN

Data is genuinely document-shaped and rarely joined. Catalogues, content, event payloads.

Key-value

Redis. Sub-millisecond lookup by a key you already know.

CHOOSE WHEN

Caching and sessions, alongside a real database. Never as the source of truth.

Analytical

ClickHouse, BigQuery. Aggregating billions of rows fast.

CHOOSE WHEN

Reporting has outgrown what your main database can serve comfortably.

Search

Elasticsearch, Meilisearch. Fuzzy text and relevance ranking.

CHOOSE WHEN

Users search free text and “close enough” results must rank sensibly.

Embedded

SQLite. A database inside the application, no server.

CHOOSE WHEN

The software must work with no network — a store during an internet outage.

PostgreSQL at the centre, SQLite in each store. Two databases, and the second one exists for exactly one reason: a store cannot stop selling because the internet did.

What changes when AI enters technology choice

AI IS GENUINELY GOOD AT THIS

- Comparing options honestly if you give it your real constraints
- Explaining an unfamiliar framework fast enough that evaluating three is now realistic
- Writing the first working integration with a library you have never used
- Generating the boring boilerplate for whatever you pick

WHERE IT QUIETLY GETS IT WRONG

- Ask it “what should I use?” with no context and you get the internet's average answer — which is whatever most blog posts describe, not what fits your team
- It has no idea you have three engineers, a fixed budget, and a client in Thanjavur
- It is confidently current about versions and libraries that changed last month

SO YOUR JOB BECOMES

Feeding it your constraints, not asking for its opinion. Give it the four questions from two slides ago and it becomes a genuinely useful analyst. Ask it cold and it will cargo-cult on your behalf, faster than you ever could manually.



NEXTGEN BUILDER INITIATIVE

presented by Aadhirai Innovations



DAY 36 — 50 · 00:44 — 01:00

Architecture

Deciding the shape — the most misunderstood decision in software

IN THIS STAGE

What architecture means

Four shapes

When to choose which

AI at this stage

1

Gather

2

Design

3

Tech

4

Architect

5

Build

6

Test

7

Deploy

8

Run

Not the code. The shape.



IN PLAIN ENGLISH

Think of a restaurant kitchen. One big kitchen where everyone cooks everything? Or separate stations — grill, fry, dessert — each with its own staff and supplies? Same food comes out. Completely different problems when it gets busy, when someone quits, or when the fryer breaks.

Architecture is that choice, for software. It decides four things:

Who can work

How many people can change things at once without colliding.

What breaks together

When one part fails, what else stops working.

What it costs

To run every month, and to keep somebody employed to operate it.

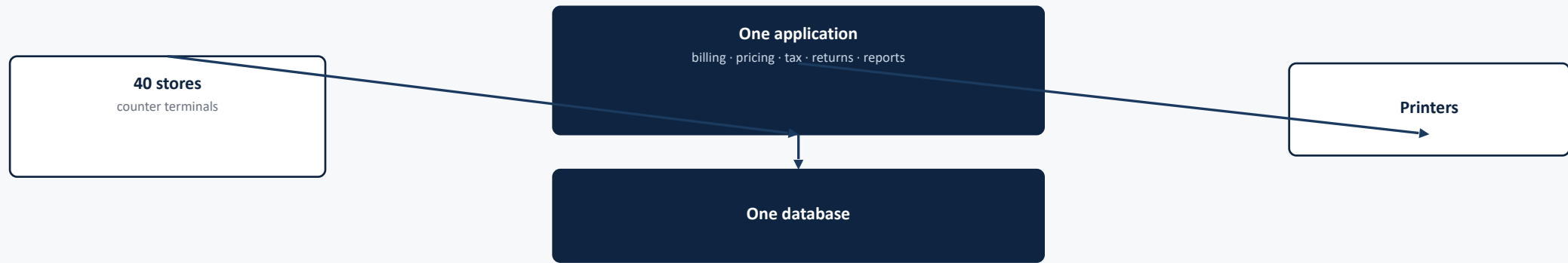
How hard it is to change

In year two, when the requirement changes — and it will.

It is not about which is modern. Every shape below is somebody's correct answer. The skill is knowing whose.

The most common mistake in the industry is treating this as a ladder — monolith for beginners, microservices for the advanced. It is not a ladder. They are answers to different problems.

One program, one database



WHERE IT WINS

- Fastest to build and to understand
- One place to look when something breaks
- Correctness is easy — one database, one transaction
- Cheapest to run

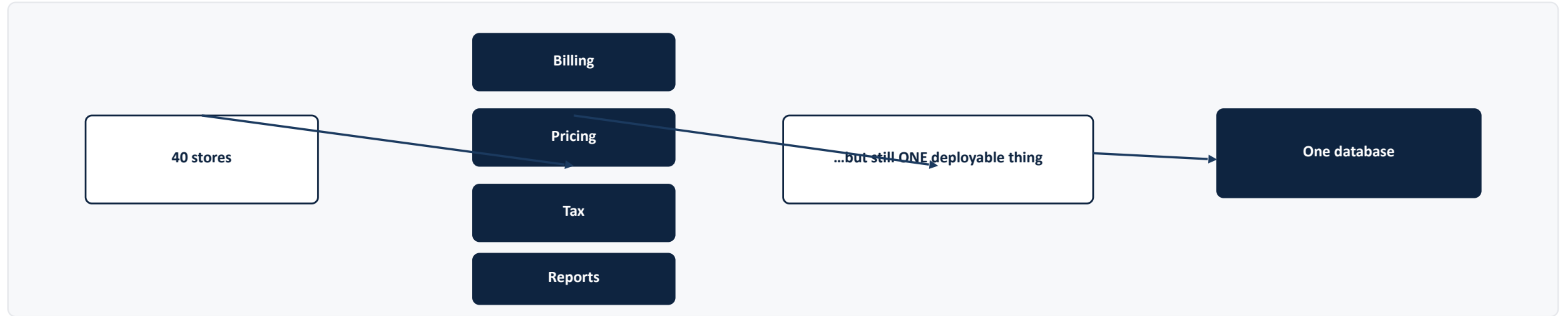
WHERE IT HURTS

- One bad deploy takes down all 40 stores
- Everything scales together or not at all
- A big team steps on each other
- Stops working the moment the internet does

CHOOSE THIS WHEN

A small team, a system you need working soon, and no part that needs to scale differently from the rest. Far more often correct than the internet suggests.

One program, clean internal walls



WHERE IT WINS

- All of A's simplicity and correctness
- Walls stop the code turning to mud
- You can split it later if you ever need to
- One deploy, one log, one place to look

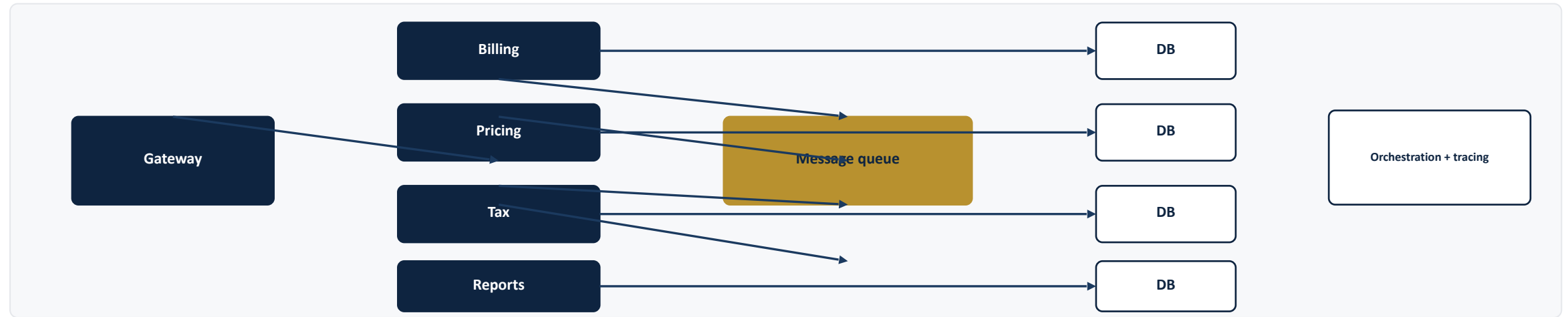
WHERE IT HURTS

- The walls only exist if people respect them
- Still one deploy — a bad change affects everything
- Requires discipline that no tool enforces for you

CHOOSE THIS WHEN

You want a monolith but expect the system to live for years and change hands. This is the sweet spot for most business software.

Many small programs



WHERE IT WINS

- Teams ship without waiting for each other
- One service failing need not stop the rest
- Scale only the part under load
- Walls are enforced by the network, not by trust

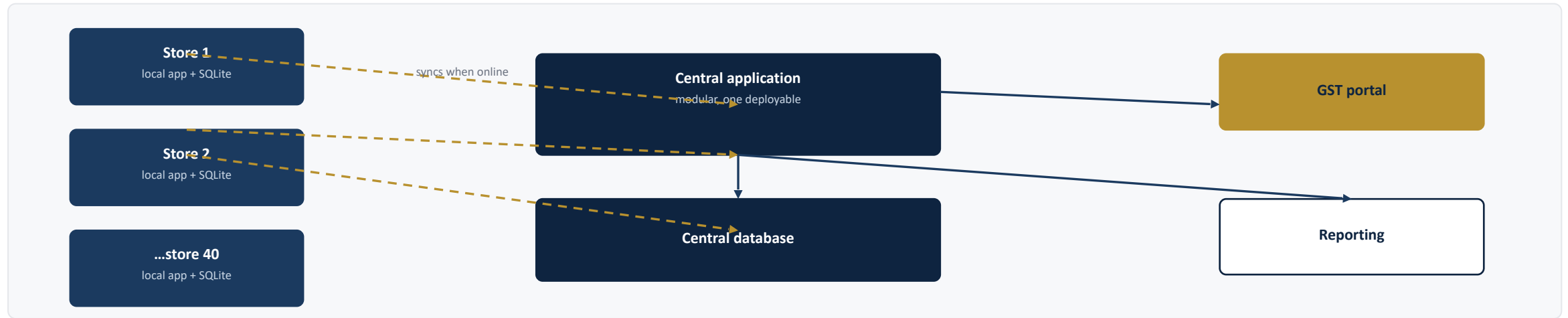
WHERE IT HURTS

- A bill touching pricing and tax now spans three databases — correctness gets genuinely hard
- 3–5× the running cost for the same features
- One bug becomes a four-service investigation
- Needs skills and headcount most teams do not have

CHOOSE THIS WHEN

Multiple teams genuinely block each other on releases, or one part must scale very differently. Not before that. This is the shape most often chosen for the wrong reason.

Central system, plus a copy in every store



WHERE IT WINS

- A store keeps billing when the internet dies — the hard requirement
- Central system stays simple: one app, one database
- Correctness lives in one transaction, centrally
- Fits the budget with room left over

WHERE IT HURTS

- Two things to build and keep in step
- Sync and conflict resolution is the hardest code in the project
- A store can be briefly out of date, and staff must be able to see that

CHOOSE THIS WHEN

One hard constraint — here, stores that cannot stop selling — makes the simple central design impossible on its own. Solve it exactly where it exists, not everywhere.

The five things that actually decide it

How big is the team?	3–8 people	A small team cannot operate many services. Kills C.
How often does each part change?	All together	Nothing changes at a different rate. No reason to split.
What must never be wrong?	Bill numbers, stock	One database makes this free. Splitting makes it hard.
What may fail together?	Not a store	A store must keep billing alone. Forces the local copy.
What can it cost to run?	Fixed, modest	C costs 4× for identical features.

WE CHOOSE D — AND HERE IS WHAT WE GIVE UP

Independent deploys. Independent scaling. If this chain grows to 400 stores with six engineering teams, we revisit — and that is fine, because we write down today why we chose it, and what would change our mind.

What changes when AI enters architecture

AI IS GENUINELY GOOD AT THIS

- Exploring four designs in an afternoon instead of a week
- Drawing the diagrams and building the comparison table for you
- Playing devil's advocate: “attack this design, find where it fails”
- Drafting the decision record once you have decided

WHERE IT QUIETLY GETS IT WRONG

- Ask “what architecture should I use?” and you get microservices — because that is what most articles describe, not what fits you
- It cannot know you have four engineers, a fixed budget, and one shop with bad internet
- It will produce a beautiful, confident, over-engineered design and give you no signal that it is wrong

SO YOUR JOB BECOMES

Making the call, and owning it. AI can lay out every option faster than any human. It cannot decide which constraint matters most to your client — and it will never be the one explaining the outage.



NEXTGEN BUILDER INITIATIVE

presented by Aadhirai Innovations



DAY 51 — 130 · 01:00 — 01:08

Build

Eighty days of writing code — and the part AI changed most

IN THIS STAGE

How work actually flows

Branching and review

AI at this stage

What juniors do now

1

Gather

2

Design

3

Tech

4

Architect

5

Build

6

Test

7

Deploy

8

Run

The parts nobody shows you in college

DAY 55 • YOUR FIRST PULL REQUEST

You finish the feature in a day. It takes four days to get merged. Nobody told you that reviewing is most of the job.

1

Pick up a story

From the board, with acceptance criteria already written

2

Branch

Your own copy. Nobody else's work is affected

3

Write + test

The test is not optional. It is how the next person changes your code safely

4

Pull request

You ask two people to read it. This is where you learn fastest

5

Review

Comments, changes, arguments. Not personal — this is the quality gate

6

Merge

It joins the main line. The pipeline verifies everything still works

In a real team you will read far more code than you write. That is not a waste of your time — it is the job.

For students

Contribute to anything with real code review. It compresses two years of learning into six months.

For mid-level

Your review comments are how you scale. Teaching three people beats out-coding them.

What changes when AI enters writing code

AI IS GENUINELY GOOD AT THIS

- Writing boilerplate, CRUD, mappers and glue in seconds
- Explaining an unfamiliar codebase or library faster than any documentation
- First-draft tests, first-draft migrations, first-draft everything
- Turning a stack trace into a plausible explanation in one step

WHERE IT QUIETLY GETS IT WRONG

- Concurrency. Two counters billing the last item is exactly the code AI most often gets subtly wrong — and it looks fine
- Queries that work on 10 rows and die on 10 million. It has never seen your data volume
- It writes code that compiles and passes. Neither means correct
- You will approve more code per hour than you can genuinely understand

SO YOUR JOB BECOMES

Judging, not typing. The bottleneck moved from writing code to verifying it. If you cannot explain a line you shipped, you did not write it — you accepted it. And you own it either way.



NEXTGEN BUILDER INITIATIVE

presented by Aadhirai Innovations



DAY 90 — 150 · 01:08 — 01:16

Test

Which tests, at which stage, and the three that catch disasters

IN THIS STAGE

Types of testing

Which stage each belongs to

The three that matter

AI at this stage

1

Gather

2

Design

3

Tech

4

Architect

5

Build

6

Test

7

Deploy

8

Run

Six kinds. Each catches something the others cannot.

Unit

One function alone. Does the tax calculation give the right number?

CHOOSE WHEN

Always. Runs on every save. Seconds. This is where logic errors die.

Integration

Your code plus a real database. Does the transaction actually hold?

CHOOSE WHEN

Always. This is where wrong assumptions about the database surface.

End-to-end

The whole system through the real screen, like a user.

CHOOSE WHEN

A handful only. Slow and brittle — but catches wiring nothing else sees.

Performance / load

Does it survive the busiest hour, with five years of data?

CHOOSE WHEN

Before release. Test on real data volume, never on an empty database.

Security

Vulnerable code, vulnerable libraries, vulnerable running app.

CHOOSE WHEN

Dependency scanning on every commit — highest value per minute you will ever spend.

User acceptance

The client's own staff try to do their real job with it.

CHOOSE WHEN

Before go-live. The only test that catches a misunderstood requirement.

Which stage? Unit and integration on every commit. Security scanning on every commit. End-to-end before release. Load and UAT before go-live. Smoke test after every deploy.

If you could only ever write three

01

Race for the last item

20 threads. One unit of stock. Assert exactly one succeeds.

Two counters selling the same last item. Silent, and you find out at stock-take.

02

Race for the bill number

20 threads bill at once. Assert no gaps, no duplicates.

A duplicate GST bill number is a legal problem, not a bug report.

03

Does the ledger add up?

Assert today's stock equals the sum of everything that moved.

Catches every kind of quiet drift between what you stored and what happened.

All three protect something that would be catastrophic AND completely silent if it were wrong. That is the filter — not coverage percentage.

What changes when AI enters testing

AI IS GENUINELY GOOD AT THIS

- Generating a full test suite for existing code in under a minute
- Suggesting edge cases you genuinely would not have listed
- Creating realistic test data and fixtures at volume
- There is no longer any excuse for an untested code path

WHERE IT QUIETLY GETS IT WRONG

- Generated tests pass. That is the danger. A test that calls the function, catches everything and asserts nothing goes green forever while the behaviour rots
- Coverage climbs, confidence climbs, and neither is earned
- It tests the happy path. The cancelled bill, the half return, the expired batch — those still come from someone who has seen them go wrong

SO YOUR JOB BECOMES

Reading every assertion. A false green is worse than no test at all, because it buys confidence you did not pay for. If a generated test would still pass after you break the function, delete it.



NEXTGEN BUILDER INITIATIVE

presented by Aadhirai Innovations



DAY 151 — 165 · 01:16 — 01:24

Deploy

Getting it to 40 stores without breaking any of them

IN THIS STAGE

Where software lives

How to release safely

The pipeline

AI at this stage

1

Gather

2

Design

3

Tech

4

Architect

5

Build

6

Test

7

Deploy

8

Run

Five options. Very different lives.

Own server

A physical machine the client buys and keeps in the office.

CHOOSE WHEN

Data must never leave the premises, or there is genuinely no internet. Somebody on site must be able to swap a disk.

Rented VM

A virtual machine you rent and administer yourself.

CHOOSE WHEN

Cheapest cloud option and you have the skill to patch, back up and monitor it yourself.

Managed platform

Push your code; the platform runs and patches it.

CHOOSE WHEN

Small team, nobody wants to be a sysadmin. The right default for most business systems.

Kubernetes

A cluster that schedules and heals containers automatically.

CHOOSE WHEN

Many services, many teams, real scaling needs. Powerful, and genuinely expensive in expertise.

Serverless

Code runs per request; nothing exists in between.

CHOOSE WHEN

Spiky or occasional work — webhooks, nightly jobs, image processing. Costs invert at steady load.

Local at the edge

A copy running in each store, syncing back.

CHOOSE WHEN

The site cannot stop working when the network does. Hard to build — use only where required.

Managed platform at the centre, a local copy in each store. The second one exists for exactly one reason — and it is a reason from the business, not from engineering.

Four ways to put new code in front of users

Stop and restart

Turn the old version off, turn the new one on. Brief downtime.

CHOOSE WHEN

Downtime is acceptable — deploying at 11 p.m. when no store is billing. Simplest thing that works.

Rolling

Replace instances one at a time. No downtime.

CHOOSE WHEN

You run several copies and cannot afford any downtime at all.

Blue-green

Two full environments. Switch traffic. Switch back instantly.

CHOOSE WHEN

Rollback must be instant and you can afford to run two of everything during release.

Pilot / canary

Two stores get the new version first. Then twenty. Then all forty.

CHOOSE WHEN

The blast radius of a bad release is large. This is how 40-store rollouts should always work.

WHAT WE DO

Two pilot stores for a week. Then ten. Then the rest. Because the first real Deepavali rush is not the moment to discover the bug — and because a store manager who was warned is an ally, while one who was surprised is an enemy.

And one rule that matters more than all four

Build the thing once. Test that exact thing. Deploy that exact thing. Never rebuild for production — a rebuild is a different artifact that no test has ever run against, and then “it worked in testing” means nothing.

From your laptop to 40 stores, automatically

?

IN PLAIN ENGLISH — WHAT IS CI/CD?

A factory line for your code. Every time anyone changes something, it goes down the line and gets checked at each station. If any check fails, it never reaches a customer. The point is not speed — it is that the path to production is identical every single time, so the deploy is never the thing you have to suspect.



Steps 1–5 are free

They run on every change, take four minutes, and catch most mistakes before a human sees them.

Step 11 is a person

Bills are legal documents. Somebody approves the release. Not every system needs this — this one does.

Step 15 needs nobody

Smoke test fails, previous version comes back automatically. No decision at 11 p.m.

What changes when AI enters deployment and pipelines

AI IS GENUINELY GOOD AT THIS

- Writing pipeline configuration from a plain description in minutes
- Explaining unfamiliar YAML, Docker or cloud syntax instantly
- Generating infrastructure-as-code for a setup you have described
- Spotting drift between what your config says and what is actually running

WHERE IT QUIETLY GETS IT WRONG

- Generated pipelines quietly grant themselves broad admin permissions to make things work
- It will disable a check that was failing rather than tell you the check was right
- Hardcoded secrets in environment blocks appear more often than anyone admits
- Cloud costs: it will happily suggest a setup that works and costs ten times what you needed

SO YOUR JOB BECOMES

Reading the permissions line by line. Generated infrastructure is the fastest way to hand somebody your production credentials. The code works — that was never the hard part.



NEXTGEN BUILDER INITIATIVE

presented by Aadhirai Innovations



DAY 166 + · 01:24 — 01:32

Run

Monitoring, the 9 p.m. phone call, and handing it over

IN THIS STAGE

What to watch

What to alert on

Handover

AI at this stage

1

Gather

2

Design

3

Tech

4

Architect

5

Build

6

Test

7

Deploy

8

Run

Alert on what a person feels, not what a machine reports

DAY 170 · 9:12 P.M.

A store manager phones. "Billing is slow." Every dashboard is green. CPU is fine. No errors. Nothing has crashed. And nobody can pay.

ALERTS PEOPLE LEARN TO IGNORE

- CPU above 80%
- Memory above 70%
- Disk at 60%
- A process restarted
- An error appeared in a log

None of these means a customer is stuck. Wake someone twice for these and they mute the channel.

ALERTS WORTH WAKING UP FOR

- Zero bills in a store for 20 minutes while it is open
- Bill printing slower than 5 seconds
- A store has not synced in an hour
- Error rate above 2% for five minutes
- Backup did not finish

Every one means a real person cannot do the thing they came to do.

Health checks tell you the process is running. They do not tell you the business is working.

It is not finished when it works

It is finished when somebody else can run it, change it and recover it.

A BAD HANDOVER

- “It just works, don't touch it”
- Deploy runs from one person's laptop
- Accounts in the developer's name
- No rollback has ever been tried
- Docs written in the last week from memory

WHAT YOU ACTUALLY HAND OVER

- Runbook — deploy, roll back, restore, rotate a key
- Decision records — why it is shaped this way
- Access list — every account, in the client's name
- Known issues — the fragile parts, honestly
- Who to call, tested by actually calling them

THE HANDOVER IS NOT THE DOCUMENTS. IT IS THE REHEARSAL.

The next developer does one deploy, one rollback and one restore themselves, on a quiet afternoon, while you watch without touching the keyboard. Every step they get stuck on is a defect in your runbook, fixed on the spot. Documents lie. A rehearsal cannot.

What changes when AI enters running it in production

AI IS GENUINELY GOOD AT THIS

- Summarising thousands of log lines into a plausible story in seconds
- Spotting correlations across services a human would take an hour to find
- Drafting the incident timeline while the incident is still happening
- Writing the post-mortem while the details are fresh

WHERE IT QUIETLY GETS IT WRONG

- It will confidently propose a root cause built from correlation, not evidence
- Rolling back the wrong thing during a live incident makes everything worse
- It does not know which of your stores is having a festival sale tonight

SO YOUR JOB BECOMES

Deciding what changes in production. Let AI narrow the search — it is genuinely excellent at that. A human decides what to actually do, because a human is the one who will explain it tomorrow.

180 days on one slide



Gather	Five techniques. Watch people work — it finds what interviews cannot.
Design	Functional decides the screens. Non-functional decides the architecture.
Tech	Choose for who you can hire in 2032, not for milliseconds.
Architect	Four shapes. The hard constraint picks one. Write down what you gave up.
Build	You will read more code than you write. That is the job now.
Test	Three tests protect what is catastrophic and silent.
Deploy	Build once, deploy that exact thing. Pilot before you roll out.
Run	Alert on what a person feels. Rehearse the handover.

Almost every decision in this story was a decision NOT to add something. That is the skill.

And the one thing it did not

WHAT GOT FASTER

- Drafting the requirement — a week became an afternoon
- Comparing four architectures — days became hours
- Writing the code — the boilerplate is gone
- Generating tests, pipelines, infrastructure
- Reading logs during an incident

WHAT DID NOT MOVE AT ALL

- Sitting at a counter and noticing the notebook
- Deciding which constraint matters most
- Knowing that a number was invented
- Owning the outage and explaining it
- Saying no to something that should not be built

The bottleneck moved from producing to judging. Every stage now takes less time and more responsibility.

If you are a student

Learn to read code critically, not just write it. That is now the scarce skill.

If you are mid-level

Your value moved to judgment — what to build, what to refuse, what to check.



NEXTGEN BUILDER INITIATIVE

presented by Aadhirai Innovations

01:36 — 02:00 · YOUR TURN

Bring your own project

The most useful part of this session is the part I have not prepared.

Students

“Here is my final-year project idea — what am I not thinking about?”

Mid-level

“We are about to split into microservices. Should we?”

Anyone

“Here is a decision at work I disagree with. How would you argue it?”



NEXTGEN BUILDER INITIATIVE

presented by Aadhirai Innovations

Thank you

Slides, the recording and your certificate arrive within 24 hours.

NEXT IN THE SERIES

The 4-hour hands-on workshop — we take this same requirement and actually build it, deploy it live, break it on purpose, and fix it while you watch. Bring a laptop. Details in your email.

Feedback form

Two minutes. It decides what we run next.

Your certificate

By email within 24 hours.

One email a month

Sessions only. Nothing else.